INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

A Project Report on

SmartShop

An Online Product Store

Course Code: CSE-3532

Course Title: Tools and Technologies for Internet Programming

Submitted By

Tafsirul Islam (C243025)

Rafi Alam (C243015)

Hasan Al Faozan (C243048)

Submitted To

Name: Nazim Uddin

Department of CSE, IIUC

Date of Submission: 30th Oct 2026

Live website : *https://smart-shop-two-fawn.vercel.app*

GitHub Repository: https://github.com/tafsirulislamapon/smart-shop.git

Abstract

SmartShop is a responsive e-commerce web application developed as the mid project of the course CSE-3532, Tools and Technologies for Internet Programming. The website lets users browse products, manage a shopping cart and pay from a virtual balance. Product data is fetched in real time from the FakeStore REST API, and customer reviews are loaded from a local JSON file. The cart calculates the subtotal, delivery charge, shipping cost, coupon discount and final total without reloading the page. The user balance is stored in the browser with localStorage, so it remains after a page reload. As an additional feature, products can be sorted by price. A contact form with validation and an About Us section are included as bonus features.

The project is built with HTML5, Tailwind CSS and plain JavaScript only, with no frameworks and no raw CSS. All features were tested in the Google Chrome browser on desktop and mobile screen sizes, and all test cases passed.

Keywords: *e-commerce, HTML5, Tailwind CSS, JavaScript, DOM, Fetch API, REST API, JSON, localStorage*

Acknowledgement

We would like to thank our course teacher for the guidance and support given throughout this project. We are also grateful to the Department of Computer Science and Engineering, International Islamic University Chittagong, for giving us the opportunity to apply web technologies in a practical project.

Table of Contents

1. Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 1
1.5 Report Organization 1
2. Tools and Technologies **2**
3. System Design **3**
3.1 Project Structure 3
3.2 Page Layout 3
3.3 Data Flow 3
4. Implementation **5**
4.1 Navigation Bar 5
4.2 Sliding Banner 6
4.3 Product Section 7
4.4 Shopping Cart 7
4.5 User Balance System 9
4.6 Review Carousel 9
4.7 Additional Feature: Product Sorting 10
4.8 Bonus Feature: Contact Form 11
4.9 Bonus Feature: About Us 12
4.10 Footer and Back to Top 12
4.11 Responsive Design and Icons 12
5. Testing and Results **14**
5.1 Test Method 14
5.2 Test Cases 14
5.3 Results 15
6. Course Outcome (CO2) Mapping **16**
7. Challenges and Solutions **17**
8. Conclusion and Future Work **18**
8.1 Conclusion 18
8.2 Future Work 18
Appendix A: Team Contribution **19**
Appendix B: How to Run the Project **19**
References **20**

1. Introduction

1.1 Background

Online shopping is now a normal part of daily life. A typical e-commerce website shows products, keeps a shopping cart and calculates the cost of an order. Building one is a good way to practise the core web technologies: HTML, CSS and JavaScript.

1.2 Problem Statement

The task was to build a dynamic online store named SmartShop. The store must load its products from a public API, update the cart and the total cost in real time, show customer reviews and control spending with a user balance. The website must be responsive and must be built with HTML, Tailwind CSS and JavaScript only.

1.3 Objectives

The main objectives of the project are:

1. To build a well-structured web page using semantic HTML5.
2. To design a responsive layout for mobile and desktop using Tailwind CSS.
3. To fetch product data from a REST API and display it with the DOM.
4. To build a shopping cart that updates all totals without reloading the page.
5. To store the user balance in the browser using localStorage.
6. To add an additional feature (product sorting) and bonus sections (Contact and About Us).

1.4 Scope

SmartShop is a front-end application. It has no server or database of its own. Products come from a public API, reviews come from a local JSON file and the balance is stored in the browser. Orders are simulated, so no real payment is made.

1.5 Report Organization

Chapter 2 covers the tools used, Chapter 3 the system design and Chapter 4 the implementation. Chapter 5 presents the testing, Chapter 6 the CO2 mapping, Chapter 7 the challenges and Chapter 8 the conclusion.

2. Tools and Technologies

The project follows the technical rules of the course: only HTML, Tailwind CSS and JavaScript are used. No JavaScript framework such as React or Angular is used, and no raw CSS is written. All styling is done with Tailwind utility classes [2]. Table 2.1 lists every tool and its purpose in the project.

Table 2.1: Tools and technologies used in SmartShop

Tool / Technology	Purpose in the Project
HTML5	Page structure with semantic tags such as header, nav, section, aside and footer
Tailwind CSS (CDN)	All styling and the responsive layout
JavaScript (ES6)	All interactive features and calculations
DOM API	Reading and changing page content in real time
Fetch API	Loading products from the API and reviews from the JSON file [4]
FakeStore API	Public REST API that provides product data in JSON format [1]
localStorage	Saving the user balance in the browser [5]
Google Fonts (Inter)	Clean and readable typeface for the website [12]
Heroicons	SVG icons for arrows, menu, plus, stars and back to top [3]
Visual Studio Code, Live Server	Code editor and local development server
Git and GitHub	Version control and project hosting

3. System Design

3.1 Project Structure

The project is kept small so that every part is easy to understand. It has three main files, shown in Table 3.1. The HTML file holds the layout, the JavaScript file holds all logic, and the JSON file holds the reviews.

Table 3.1: Project files

File / Folder	Description
index.html	The complete page layout, styled with Tailwind classes
script.js	All JavaScript code, divided into parts by feature
reviews.json	Six customer reviews with name, comment, rating and date
assets/	Reference images used in the README file
README.md	Short project guide for the GitHub repository

3.2 Page Layout

SmartShop is a single-page website. Each part of the page is an HTML section with its own id, so the navigation links can jump to it. Table 3.2 shows the sections in the order they appear.

Table 3.2: Sections of the web page

Section (id)	Content
header	Sticky navigation bar with menu links and cart counter
home	Sliding promotional banner
products	Sort menu, product cards and the shopping cart with balance
reviews	Customer review carousel
about	Information about SmartShop and the team members
contact	Contact form with validation
footer	Contact information, quick links and copyright text

3.3 Data Flow

Figure 3.1 shows how data moves through the application. Data comes from three sources: the FakeStore API, the local reviews.json file and the user's actions. JavaScript keeps this data in simple arrays and variables. Whenever the data changes, a function updates the

related part of the page through the DOM. The balance is the only value that is also written back to the browser storage, so that it is not lost on reload.

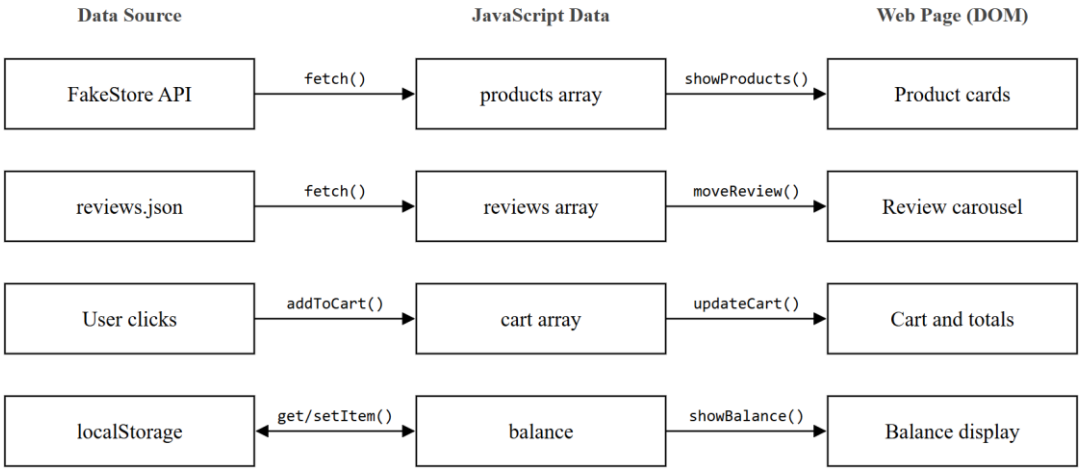


Figure 3.1: Data flow of the SmartShop application

4. Implementation

This chapter explains how each feature was implemented. Short code listings are taken directly from script.js.

4.1 Navigation Bar

The navigation bar contains five links (Home, Products, Reviews, About and Contact) and a Cart button that shows the number of items in the cart. It uses the Tailwind classes sticky and top-0, so it stays visible at the top of the screen while scrolling. On small screens the links are hidden and a menu icon button shows or hides them with classList.toggle().

The active link is highlighted dynamically. On every scroll event, the code compares window.scrollY, the distance the page has scrolled [9], with the offsetTop of each section. The last section that has been reached becomes the active one. When the bottom of the page is reached, which is checked with window.innerHeight and document.body.offsetHeight [10], [11], the Contact link is highlighted. Listing 4.1 shows this logic.

Listing 4.1: Highlighting the active menu link

```
document.querySelectorAll("section").forEach((section) => {
  if (window.scrollY >= section.offsetTop - 100) {
    currentSection = section.id;
  }
});

const atBottom = window.innerHeight + window.scrollY >= document.body.offsetHeight - 5;
if (atBottom) currentSection = "contact";

document.querySelectorAll(".nav-link").forEach((link) => {
  link.classList.toggle("bg-yellow-400",
    link.getAttribute("href") === "#" + currentSection);
});
```

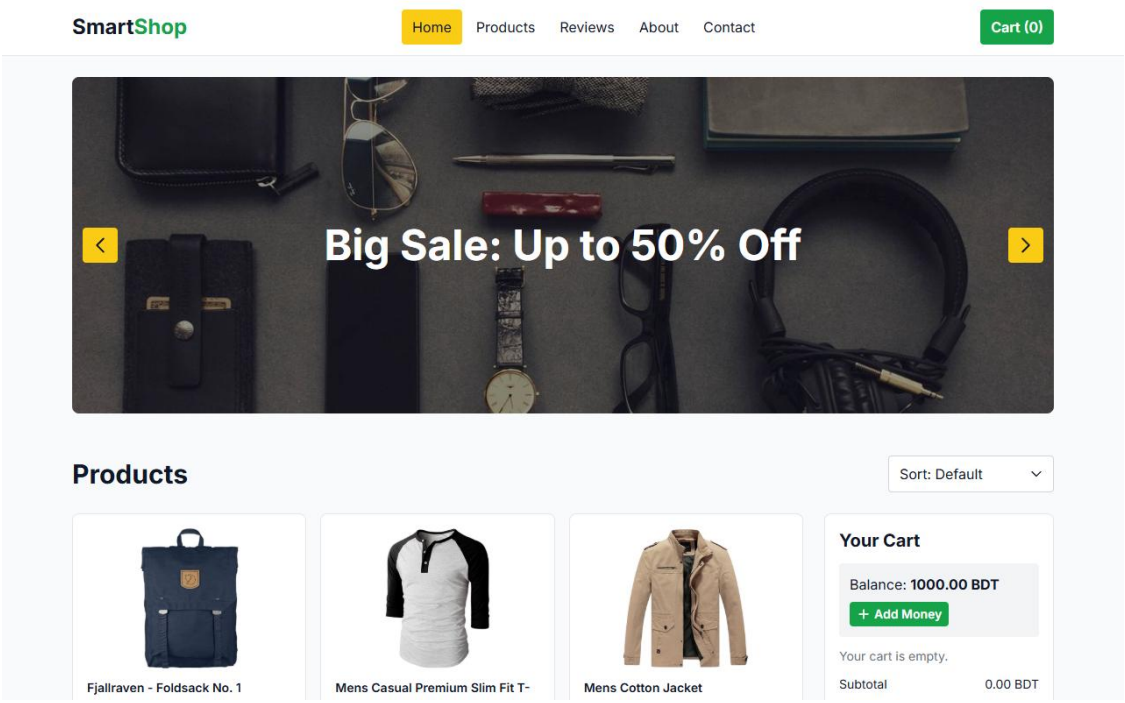


Figure 4.1: Home page with the sticky navigation bar and banner

4.2 Sliding Banner

The banner shows four promotional slides. Each slide is an object with an image and a text, stored in the banners array. The function moveBanner() changes the current index by a step of 1 (next) or -1 (previous). If the index goes past the last slide it returns to the first one, and if it goes before the first slide it moves to the last one. The slides change automatically every three seconds with setInterval(), and they can also be changed manually with the arrow buttons. The images are provided by Lorem Picsum [13].

Listing 4.2: Moving the banner

```
function moveBanner(step) {
  bannerIndex = bannerIndex + step;
  if (bannerIndex >= banners.length) bannerIndex = 0;
  if (bannerIndex < 0) bannerIndex = banners.length - 1;
  document.getElementById("bannerImage").src = banners[bannerIndex].image;
  document.getElementById("bannerText").textContent = banners[bannerIndex].text;
}

moveBanner(0);
setInterval(() => moveBanner(1), 3000);
```



Figure 4.2: A banner slide with the previous and next buttons

4.3 Product Section

Products are loaded from the FakeStore API [1] with the Fetch API [4], as shown in Listing 4.3. The response is converted to a JavaScript array with response.json() and saved in the products variable. The function showProducts() then loops over the array with forEach() and builds an HTML card for each product. Each card shows the product image, title, rating, price and an Add to Cart button. Long titles are limited to two lines with the Tailwind class line-clamp-2. Prices are shown in BDT so that they work with the balance system.

Listing 4.3: Fetching products from the API

```
fetch('https://fakestoreapi.com/products')
  .then(response => response.json())
  .then(data => {
    products = data;
    showProducts(products);
  })
  .catch(() => {
    document.getElementById("productList").textContent = "Could not load products.";
  });
```

4.4 Shopping Cart

The cart is an array of items. Each item stores the product id, title, price and quantity. When a product is added for the first time, it is pushed into the array with a quantity of 1. If it is already in the cart, only its quantity is increased. An item is removed with filter(), which creates a new array without that item. After every change, updateCart() draws the cart again and updates all amounts on the page, so the page never reloads.

All amounts are calculated in getCartTotals() according to the rules in Table 4.1. A discount of 10% is applied when the user enters the coupon code SMART10.

Table 4.1: Rules for the cart calculation

Amount	Rule
Subtotal	Sum of price x quantity for all items
Delivery Charge	50 BDT when the cart is not empty
Shipping Cost	30 BDT when the cart is not empty
Discount	10% of the subtotal when coupon SMART10 is applied
Total	Subtotal + Delivery Charge + Shipping Cost - Discount

Listing 4.4: Calculating the cart totals

```
let subtotal = 0, delivery = 0, shipping = 0, discount = 0;

cart.forEach((item) => {
  subtotal = subtotal + item.price * item.quantity;
});

if (cart.length > 0) {
  delivery = 50;
  shipping = 30;
}
if (couponApplied) {
  discount = subtotal * 0.1;
}
const total = subtotal + delivery + shipping - discount;
```

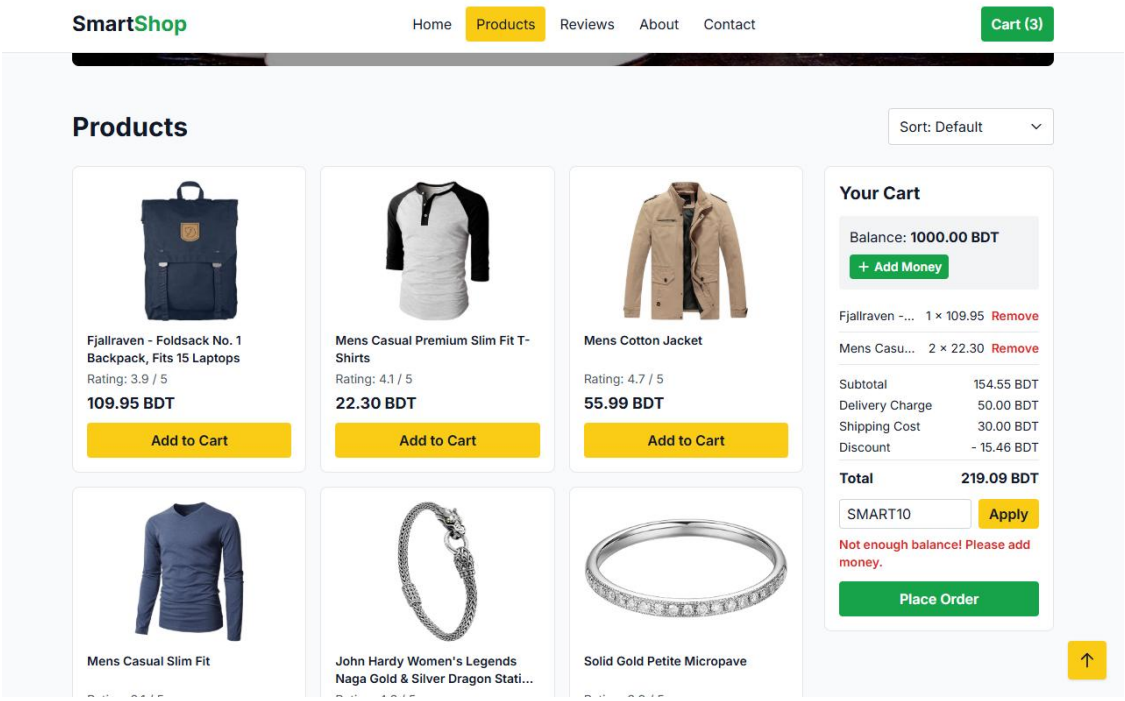


Figure 4.3: Cart with the SMART10 coupon applied and a low balance warning

4.5 User Balance System

Every user starts with a balance of 1000 BDT. The balance is saved with localStorage.setItem() each time it changes, and it is read with localStorage.getItem() when the page loads [5]. Since localStorage stores only text, the unary plus operator (+) is used to convert the saved value into a number [8]. On the first visit nothing is saved yet, so the value 1000 is used instead (Listing 4.5).

Listing 4.5: Loading and saving the balance

```
let balance = +localStorage.getItem("balance") || 1000;

function showBalance() {
  document.getElementById("balance").textContent = balance.toFixed(2);
  localStorage.setItem("balance", balance);
}
```

The cart can never cost more than the available balance. When a product is added, the new total is checked. If it is larger than the balance, the product is taken out again and a warning message is shown (Listing 4.6). The Add Money button increases the balance by 1000 BDT, and the Place Order button subtracts the total from the balance and empties the cart.

Listing 4.6: Blocking a purchase that exceeds the balance

```
if (getCartTotals().total > balance) {
  if (cartItem) {
    cartItem.quantity = cartItem.quantity - 1;
  }
  else cart.pop();

  showMessage("cartMessage", "Not enough balance! Please add money.", false);
  return;
}
```

4.6 Review Carousel

Customer reviews are stored in the local file reviews.json. They are loaded with fetch() in the same way as the products. The carousel shows one review at a time with the reviewer name, comment, rating and date. It moves automatically every four seconds and can also be moved with the previous and next buttons, using the same index logic as the banner.

The rating is shown with SVG star icons [3]. Two stars are prepared in advance: a yellow one and a gray one. The yellow star is repeated as many times as the rating, and the gray star fills the remaining places up to five (Listing 4.7). For example, a rating of 4 gives four yellow stars and one gray star.

Listing 4.7: Building the star rating

```
const stars = yellowStar.repeat(review.rating) + grayStar.repeat(5 - review.rating);
```

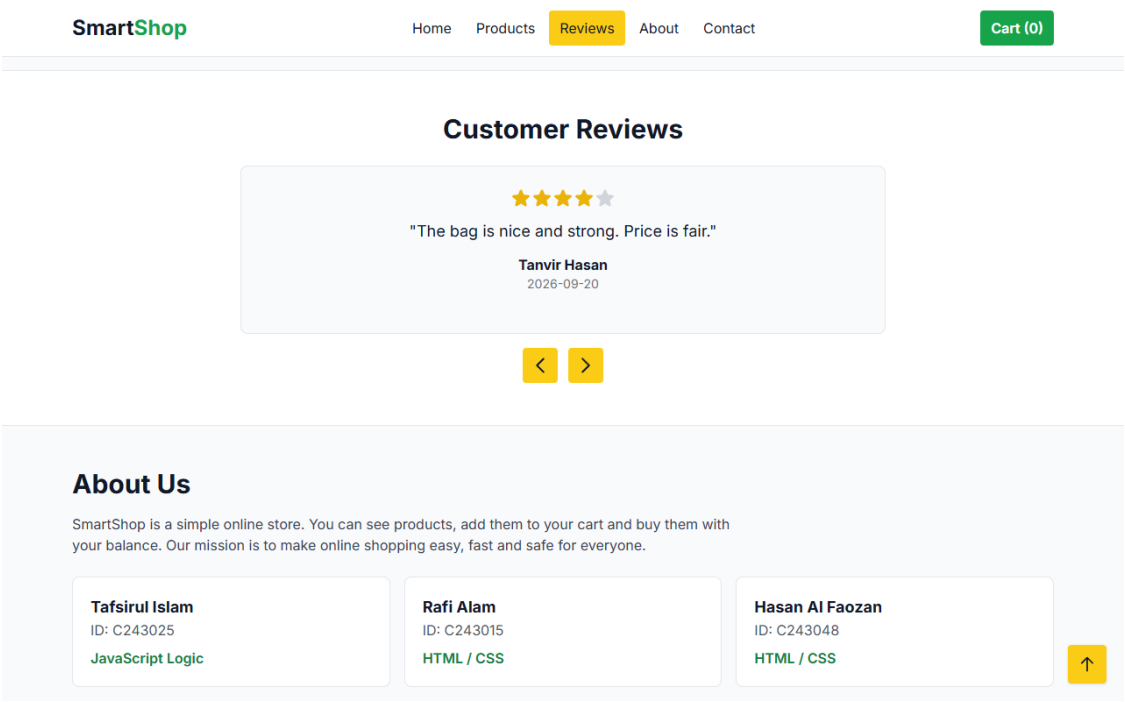


Figure 4.4: Review carousel and the About Us section

4.7 Additional Feature: Product Sorting

The additional feature chosen for this project is product sorting. A drop-down menu lets the user sort products by price from low to high or from high to low. The products array is first copied with slice(), so the original order is kept for the Default option [7]. The copy is then sorted with sort() and a compare function [6]. The function (a, b) => a.price - b.price places cheaper products first, and (a, b) => b.price - a.price places more expensive products first.

Listing 4.8: Sorting products by price

```
function sortProducts() {
  const sortType = document.getElementById("sortSelect").value;
  const sortedProducts = products.slice();

  if (sortType === "low") {
    sortedProducts.sort((a, b) => a.price - b.price);
  }
  if (sortType === "high") {
    sortedProducts.sort((a, b) => b.price - a.price);
  }
  showProducts(sortedProducts);
}
```

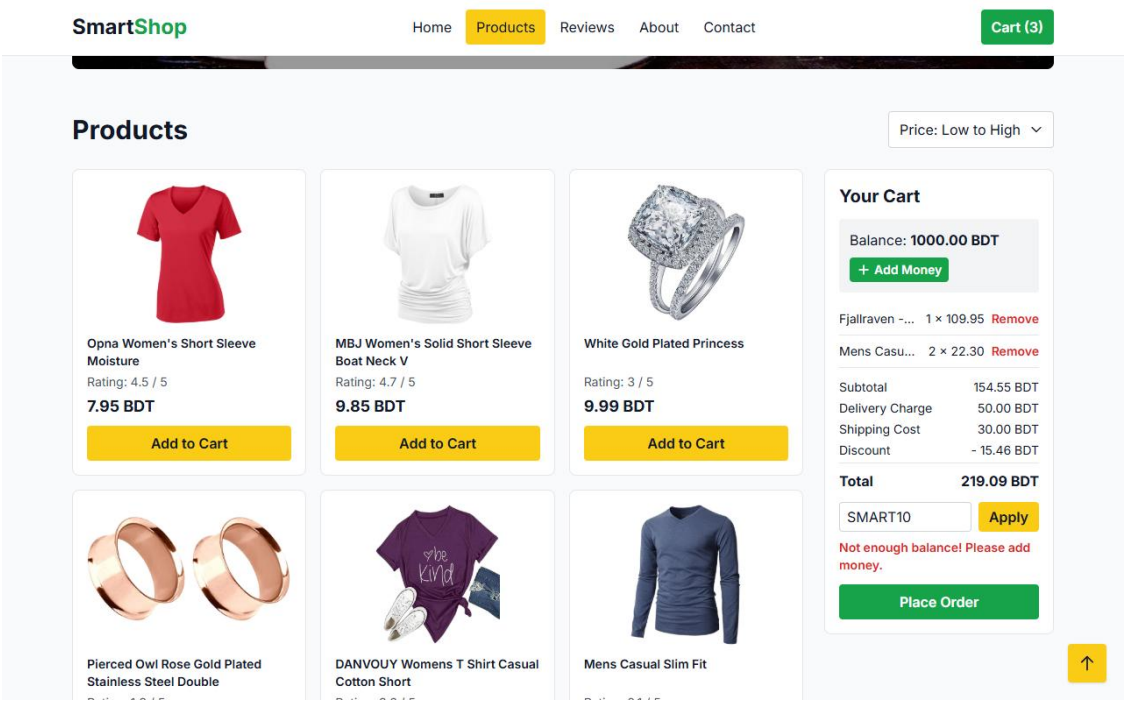


Figure 4.5: Products sorted by price from low to high

4.8 Bonus Feature: Contact Form

The contact form has three fields: name, email and message. When the form is submitted, event.preventDefault() stops the page from reloading. The input is then validated: all fields must be filled in, and the email must contain the characters "@" and ".". If the input is not valid, a red error message is shown. Otherwise, a green thank-you message with the user's name is shown and the form is cleared (Listing 4.9).

Listing 4.9: Validating the contact form

```
if (name === "" || email === "" || message === "") {
  showMessage("formMessage", "Please fill in all fields.", false);
}
else if (!email.includes("@") || !email.includes(".")) {
  showMessage("formMessage", "Please enter a valid email.", false);
}
else {
  showMessage("formMessage", "Thank you, " + name + "! We got your message.", true);
  document.getElementById("contactForm").reset();
}
```

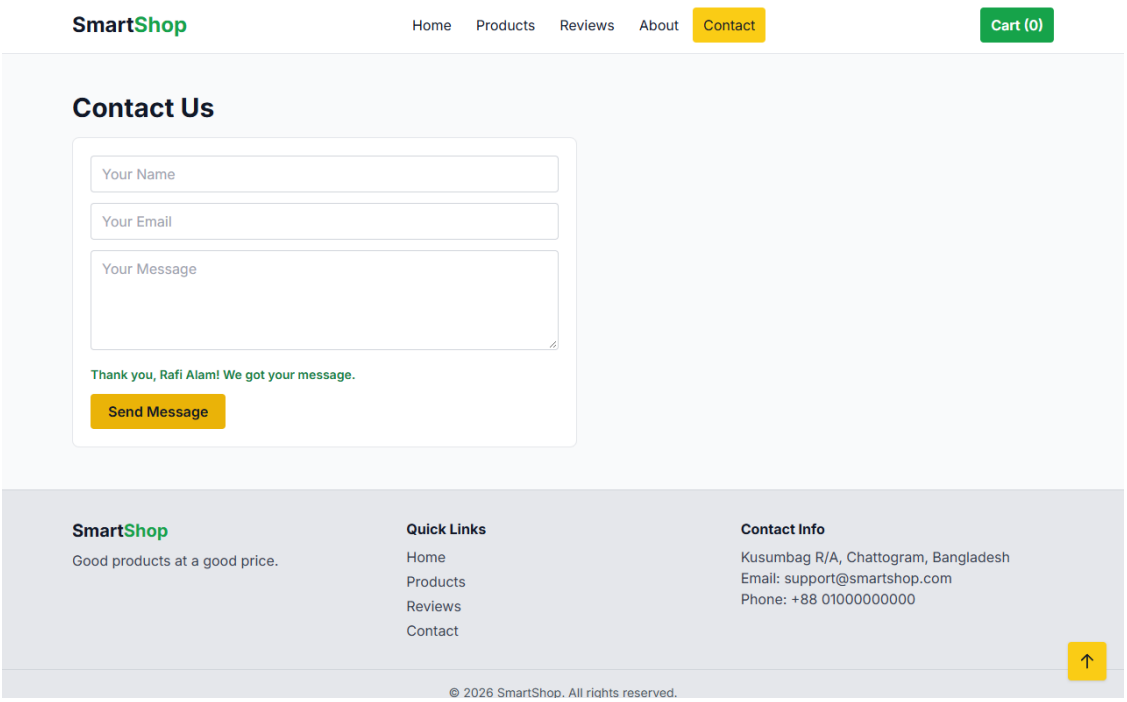


Figure 4.6: Contact form after a successful submission, with the footer

4.9 Bonus Feature: About Us

The About Us section gives a short description of SmartShop and its mission: to make online shopping easy, fast and safe for everyone. It also shows a card for each team member with name, student ID and role, as seen in Figure 4.4.

4.10 Footer and Back to Top

The footer contains the shop name, quick navigation links, contact information (Kusumbag R/A, Chattogram, Bangladesh; support@smartshop.com; +88 01000000000) and the copyright text. A Back to Top button appears in the bottom right corner after the user scrolls down 300 pixels. Clicking it calls window.scrollTo(0, 0) to return to the top of the page.

4.11 Responsive Design and Icons

The layout uses the responsive prefixes of Tailwind CSS (sm:, md: and lg:). For example, products are shown in one column on mobile phones, two columns on small screens and three columns on large screens, and the cart moves below the products on small screens. All icons are inline SVG graphics from Heroicons [3]. Because they use the current text color, their color can be changed with normal Tailwind classes. The drop-down menu uses the class appearance-none to hide the browser's default arrow, and an SVG arrow is placed in its place. Figure 4.7 shows the website on a mobile screen.

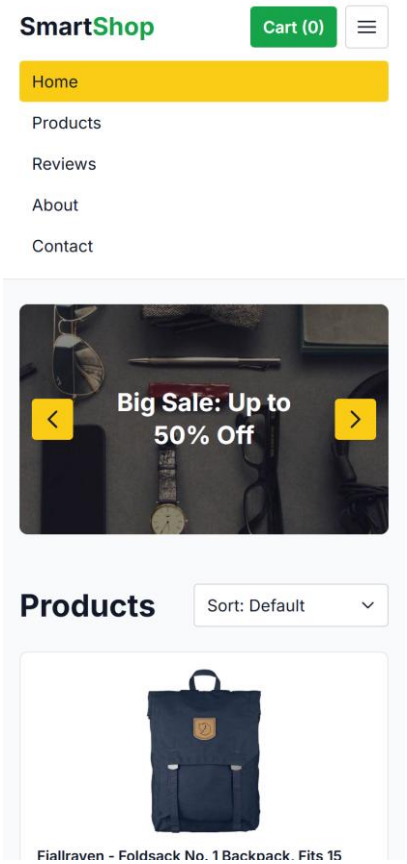


Figure 4.7: Mobile view with the menu open

5. Testing and Results

5.1 Test Method

The website was tested in the Google Chrome browser at a desktop width of 1280 pixels and a mobile width of 390 pixels. Each feature was tested with normal input and with wrong input. The test cases and their results are listed in Table 5.1.

5.2 Test Cases

Table 5.1: Test cases and results

ID	Test Case	Expected Result	Result
TC-01	Open the website	20 products are loaded from the API	Pass
TC-02	Click the next arrow on the banner	The next slide is shown	Pass
TC-03	Add one bag and two T-shirts	Subtotal 154.55 BDT, total 234.55 BDT	Pass
TC-04	Apply coupon SMART10	Discount 15.46 BDT, total 219.09 BDT	Pass
TC-05	Apply a wrong coupon code	"Invalid coupon code." is shown	Pass
TC-06	Add a 999.99 BDT item with a 1000 BDT balance	Item is not added and a warning is shown	Pass
TC-07	Click Add Money	Balance increases from 1000 to 2000 BDT	Pass
TC-08	Reload the page	Balance stays the same	Pass
TC-09	Remove an item from the cart	Item is removed and totals are updated	Pass
TC-10	Sort low to high, then high to low	First price is 7.95 BDT, then 999.99 BDT	Pass
TC-11	Click Place Order	Total is subtracted and the cart is empty	Pass
TC-12	Click next on the reviews	Next review with correct stars is shown	Pass
TC-13	Submit an empty contact form	"Please fill in all fields." is shown	Pass
TC-14	Submit the email "rafi"	"Please enter a valid email." is shown	Pass
TC-15	Submit a correct form	Thank-you message is shown	Pass
TC-16	Scroll down and click Back to Top	Page returns to the top	Pass
TC-17	Open on a 390 pixel wide screen	No horizontal scroll; menu button works	Pass

5.3 Results

All 17 test cases passed. The products load correctly from the API, the cart totals are always correct, the balance limit works as expected and the balance is kept after a reload. The layout works on both desktop and mobile screens without horizontal scrolling.

6. Course Outcome (CO2) Mapping

Course outcome CO2 requires students to apply foundational web technologies and protocols in a practical project. Table 6.1 shows how each part of SmartShop meets this outcome.

Table 6.1: Mapping of project features to CO2

Project Feature	Technology / Protocol	How It Is Achieved
Website Structure	HTML5	Semantic tags: header, nav, section, aside, footer
Responsive Design	Tailwind CSS	Responsive classes sm:, md: and lg:
Dynamic Interaction	JavaScript, DOM	innerHTML, textContent and classList update the page
Product Data Fetching	HTTP, Fetch API, REST, JSON	fetch() from the FakeStore API
Shopping Cart	JavaScript Events, DOM	Click events and real-time totals
Reviews Section	JSON, Local Data	fetch() from reviews.json
User Balance System	localStorage, JavaScript	Balance saved with localStorage.setItem()
Contact Form	HTML Forms, Validation	Input checked with JavaScript before the message
Navigation and Scrolling	Hyperlinks, DOM Events	Section links, scroll event, Back to Top

7. Challenges and Solutions

Several problems appeared during development. Table 7.1 lists each problem and how it was solved.

Table 7.1: Challenges faced and their solutions

Challenge	Solution
reviews.json did not load when index.html was opened by double click, because browsers block fetch() for local files.	The project is run with the Live Server extension of VS Code.
The sticky navigation bar covered the section titles after a menu link was clicked.	The Tailwind class scroll-mt-20 was added to every section.
Sorting changed the original order of the products.	The array is copied with slice() before sorting.
On mobile screens, the banner text went under the arrow buttons.	Side padding (px-16) was added to the banner text.
The drop-down arrow was pushed too far to the right.	The default arrow was hidden with appearance-none and an SVG arrow was added.

8. Conclusion and Future Work

8.1 Conclusion

In this project we built SmartShop, a working online store, using only HTML, Tailwind CSS and JavaScript. We learned how to fetch data from a REST API, how to update a web page with the DOM, how to keep data in the browser with localStorage and how to build a responsive layout with Tailwind CSS. The website meets all the main requirements of the project, includes the product sorting feature and adds two bonus sections. Therefore, the project fulfils course outcome CO2 of CSE-3532.

8.2 Future Work

The project can be improved in the following ways:

- Add user login and registration.
- Save the cart in localStorage so it is not lost on reload.
- Add product search and filtering by category.
- Connect a real payment gateway and a back-end database for orders.

Appendix A: Team Contribution

Table A.1: Contribution of each team member

Name	ID	Role	Contribution
Tafsirul Islam	C243025	JavaScript Logic	API integration, product cards, cart and coupon, balance system, sorting, review carousel, form validation
Rafi Alam	C243015	HTML / CSS	Page structure and responsive design with Tailwind CSS
Hasan Al Faozan	C243048	HTML / CSS	Page structure and responsive design with Tailwind CSS

Appendix B: How to Run the Project

1. Download or clone the project from the GitHub repository.
2. Open the project folder in Visual Studio Code.
3. Right click index.html and select "Open with Live Server".
4. Make sure the computer is connected to the internet, because Tailwind CSS, Google Fonts and the product API are loaded online.

References

[1] FakeStore API, "FakeStore API documentation." [Online]. Available: https://fakestoreapi.com/docs

[2] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: https://tailwindcss.com/docs

[3] Tailwind Labs, "Heroicons." [Online]. Available: https://heroicons.com

[4] MDN Web Docs, "Fetch API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[5] MDN Web Docs, "Window: localStorage property." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[6] W3Schools, "JavaScript array sort." [Online]. Available: https://www.w3schools.com/js/js_array_sort.asp

[7] W3Schools, "JavaScript array slice()." [Online]. Available: https://www.w3schools.com/jsref/jsref_slice_array.asp

[8] W3Schools, "The unary plus operator (+)." [Online]. Available: https://www.w3schools.com

[9] W3Schools, "Window scrollY property." [Online]. Available: https://www.w3schools.com/jsref/prop_win_scrolly.asp

[10] GeeksforGeeks, "JavaScript window innerWidth and innerHeight properties." [Online]. Available: https://www.geeksforgeeks.org

[11] GeeksforGeeks, "HTML DOM offsetHeight property." [Online]. Available: https://www.geeksforgeeks.org

[12] Google Fonts, "Inter." [Online]. Available: https://fonts.google.com/specimen/Inter

[13] Lorem Picsum, "Lorem Picsum: placeholder images." [Online]. Available: https://picsum.photos